

Chapter 2. Production-Readiness

While the adoption of microservice architecture brings considerable freedom to developers, ensuring availability across the microservice ecosystem requires holding individual microservices to high architectural, operational, and organizational standards. This chapter covers the challenges of microservice standardization, introduces availability as the goal of standardization, presents the eight production-readiness standards, and proposes strategies for implementing production-readiness standardization across an engineering organization.

The Challenges of Microservice Standardization

The architecture of a monolithic application is usually determined at the beginning of the application's lifecycle. For many applications, the architecture is determined at the time a company begins. As the business grows, and the application scales, developers who are adding new features often find themselves constrained and limited by the choices made when the application was first designed. They are constrained by choice of language, by the libraries they are able to use, by the development tools they can work with, and by the need for extensive regression testing to ensure that every new feature they add does not disturb or compromise the entirety of the application. Any refactoring that happens to the standalone, monolithic application is still essentially constrained by initial architectural decisions: initial conditions exclusively determine the future of the application.

The adoption of microservice architecture brings a considerable amount of freedom to developers. They are no longer tied to the architectural decisions of the past, they can architect their service however they wish, and they have free reign in decisions of language, of database, of development tools, and the like. The message accompanying the adoption of microservice architecture is usually understood and heard by developers as follows: build an application that does one thing—one thing only—and

does that one thing *extraordinarily well*; do whatever you need to do, build it however you want—just make sure it gets the job done.

While this romantic idealization of microservice development is true in principle, not all microservices are created equal—nor should they be. Each microservice is part of a microservice ecosystem, and complex dependency chains are a necessary inevitability. When you have 100, 1,000, or even 10,000 microservices, each of them will be playing a small role in a very large system. The services must interact seamlessly with one another, and—most importantly—no service or set of services should compromise the integrity of the overall system or product that they comprise. If the overall system or product is to be any good, it must be held to certain standards, and consequently, each of its parts must abide by these standards as well.

It's relatively simple to determine standards and give requirements to a microservice team if we focus on the needs of that specific team and the role their service is to play. We can say, “your microservice must do x, y, and z, and to do x, y, and z well, you need to make sure you meet this set *S* of requirements,” giving each team a set of requirements that is relevant to their service, and to their service alone. Unfortunately, this approach simply isn't scalable and neglects to recognize the important fact that a microservice is but a very small piece of an absurdly large and distributed puzzle. We must define standards and requirements for our microservices, and they must be general enough to apply to every single microservice yet specific enough to be quantifiable and produce measurable results. This is where the concept of *production-readiness* comes in.

Availability: The Goal of Standardization

Within microservice ecosystems, service-level agreements (SLAs) regarding the availability of a service are the most commonly used methods of measuring a service's success: if a service is highly available (that is, has very little downtime), then we can say with reasonable confidence (and a few caveats) that the service is doing its job.

Calculating and measuring availability is easy. You need to calculate only three measurable quantities: *uptime* (the length of time that the microservice worked correctly), *downtime* (the length of time that the microservice was *not* working correctly), and the total time a service was operational (the sum of uptime and downtime). Availability is then the uptime divided by the total time a service was operational (uptime + downtime).

As useful as it is, availability is not in itself a principle of microservice standardization, but the goal. It can't be a principle of standardization because it gives no guidance as to how to architect, build, or run the microservice: telling developers to make their microservice more available without telling them how (or *why*) to do so is useless. Availability alone comes with no concrete, applicable steps, but as we will see in the following sections, there are concrete, applicable steps that can be taken toward reaching the goal of building an available microservice.

CALCULATING AVAILABILITY

Availability is measured in so-called nines notation, which corresponds to the percentage of time that a service is available. For example, a service that is available 99% of the time is said to have “two-nines availability.”

This notation is useful because it gives us a specific amount of downtime that a service is allowed to have. If your service is required to have four-nines availability, then it is allowed 52.56 minutes of downtime per year, which is 4.38 minutes of downtime per month, 1.01 minutes of downtime per week, and 8.66 seconds of downtime per day.

Here are the availability and downtime calculations for 99% availability to 99.999% availability:

99% availability: (two-nines)

- 3.65 days/year (of allowed downtime)
- 7.20 hours/month
- 1.68 hours/week
- 14.4 minutes/day

99.9% availability (three-nines):

- 8.76 hours/year

- 43.8 minutes/month
- 10.1 minutes/week
- 1.44 minutes/day

99.99% availability (four-nines):

- 52.56 minutes/year
- 4.38 minutes/month
- 1.01 minutes/week
- 8.66 seconds/day

99.999% availability (five-nines):

- 5.26 minutes/year
 - 25.9 seconds/month
 - 6.05 seconds/week
 - 864.3 milliseconds/day
-

Production-Readiness Standards

The basic idea behind production-readiness is this: a production-ready application or service is one that can be trusted to serve production traffic. When we refer to an application or microservice as “production-ready,” we confer a great deal of trust upon it: we trust it to behave reasonably, we trust it to perform reliably, we trust it to get the job done and to do its job well with very little downtime. Production-readiness is the key to microservice standardization, the key to achieving availability across the microservice ecosystem.

However, the idea of production-readiness as stated isn’t useful enough on its own to serve as the exhaustive definition we need, and without further explication, the concept isn’t very helpful. We need to know exactly what requirements every service must meet in order to be deemed production-ready and to be trusted to serve production traffic in a reliable, appropriate way—a trust that can’t be given freely, but has to be earned. The requirements must themselves be principles that are true for every microservice, for every application, and for every distributed system: standardization without principle is meaningless.

It turns out that there is a set of eight principles that, when adopted together, fits these criteria. Each of these principles is quantifiable, gives rise to a set of actionable requirements, and produces measurable results. They are: *stability*, *reliability*, *scalability*, *fault tolerance*, *catastrophe-preparedness*, *performance*, *monitoring*, and *documentation*. The driving force behind each of these principles is that, together, they contribute to and drive the *availability* of a microservice.

Availability is, in some ways, an emergent property of a production-ready microservice. It emerges from building a scalable, reliable, fault-tolerant, performant, monitored, documented, and catastrophe-prepared microservice. Any one of these principles individually is not enough to ensure availability, but together they are: building a microservice with these principles as the driving architectural and operational requirements guarantees a highly available system that can be trusted with production traffic.

Stability

With the introduction of microservice architecture, developers are given freedom to develop and deploy at a very high velocity. New features can be added and deployed each day, bugs can be quickly fixed, any old technologies swapped out for the newest ones, and outdated microservices can be rewritten and the old versions deprecated and decommissioned. With this increased velocity comes increased instability, and in microservice ecosystems the majority of outages can usually be traced back to a bad deployment that contained buggy code or other serious errors. To ensure availability, we need to carefully guard against this instability that stems from increased developer velocity and the constant evolution of the microservice ecosystem.

Stability allows us to reach availability by giving us ways to responsibly handle changes to microservices. A stable microservice is one in which development, deployment, the addition of new technologies, and the decommissioning and deprecation of microservices do not give rise to instability within and across the larger microservice ecosystem. We can determine stability requirements for each microservice to mitigate the negative side effects that may accompany each change.

To mitigate any problems that may arise from the development cycle, stable development procedures can be put into place. To counteract any instability introduced by deployment, we can ensure our microservices are deployed carefully with proper staging, canary (a small pool of 2%–5% of production hosts), and production rollouts. To prevent the introduction of new technologies and the deprecation and decommissioning of old microservices from compromising the availability of other services, we can enforce stable introduction and deprecation procedures.

STABILITY REQUIREMENTS

The requirements of building a stable microservice are:

- A stable development cycle
- A stable deployment process
- Stable introduction and deprecation procedures

The details of stability requirements are covered in [*Chapter 3, Stability and Reliability*](#).

Reliability

Stability alone isn't enough to ensure a microservice's availability: the service must also be *reliable*. A reliable microservice is one that can be trusted by its clients, by its dependencies, and by the microservice ecosystem as a whole. A reliable microservice is one that has truly earned the trust that is essential and required in order for it to serve production traffic.

While stability is related to mitigating the negative side effects accompanying change, and reliability is related to trust, the two are inextricably linked. Each stability requirement also carries a reliability requirement alongside it: for example, developers should not only seek to have stable deployment processes, they should also ensure that each deployment is reliable from the point of view of one of their clients or dependencies.

The trust that reliability secures can be broken into several requirements, the same way we determined requirements for stability. For example, we can make our deployment processes reliable by making sure that our in-

tegration tests are comprehensive and our staging and canary deployment phases are successful so that every change introduced into production can be trusted not to contain any errors that might compromise its clients and dependencies.

By building reliability into our microservices, we can protect their availability. We can cache data so that it will be readily available to client services, helping them protect their SLAs by making our own services highly available. To protect our own SLA from any problems with the availability of our dependencies, we can implement defensive caching.

The last reliability requirement is related to routing and discovery. Availability requires that the communication and routing between different services be reliable: health checks should be accurate, requests and responses should reach their destinations, and errors should be handled carefully and appropriately.

RELIABILITY REQUIREMENTS

The requirements of building a reliable microservice are:

- A reliable deployment process
- Planning, mitigating, and protecting against the failures of dependencies
- Reliable routing and discovery

The details of production-ready reliability requirements are covered in [*Chapter 3, Stability and Reliability*](#).

Scalability

Microservice traffic is rarely static or constant, and one of the hallmarks of a successful microservice (and of a successful microservice ecosystem) is a steady increase in traffic. Microservices need to be built in preparation for this growth, they need to accommodate it easily, and they need to be able to actively scale with it. A microservice that can't scale with growth experiences increased latency, poor availability, and in extreme cases, a drastic increase in incidents and outages. *Scalability* is essential for availability, making it our third production-readiness standard.

A scalable microservice is one that can handle a large number of tasks or requests at the same time. To ensure a microservice is scalable, we need to know both (1) its qualitative growth scale (e.g., whether it scales with page views or customer orders) and (2) its quantitative growth scale (e.g., how many requests per second it can handle). Once we know the growth scale, we can plan for future capacity needs and identify resource bottlenecks and requirements.

The way a microservice handles traffic should also be scalable. It should be prepared for bursts of traffic, handle them carefully, and prevent them from taking down the service entirely. It's easier said than done, but without scalable traffic handling, developers can (and will) find themselves looking at a broken microservice ecosystem.

Additional complexity is introduced by the rest of the microservice ecosystem. The inevitable additional traffic and growth from a service's clients have to be prepared for. Likewise, any dependencies of the service should be alerted when increases in traffic are expected. Cross-team communication and collaboration are essential for scalability: regularly communicating with clients and dependencies about a service's scalability requirements, status, and any bottlenecks ensures that any services relying on each other are prepared for growth and for potential pitfalls.

Last but not least, the way a microservice stores and handles data needs to be scalable as well. Building a scalable storage solution goes a long way toward ensuring the availability of a microservice, and is one of the most essential components of a truly production-ready system.

The requirements of building a scalable microservice are:

- Well-defined quantitative and qualitative growth scales
- Identification of resource bottlenecks and requirements
- Careful, accurate capacity planning
- Scalable handling of traffic
- The scaling of dependencies
- Scalable data storage

The details of production-ready scalability requirements are covered in [*Chapter 4, Scalability and Performance*](#).

Fault Tolerance and Catastrophe-Preparedness

Even the simplest of microservices is a fairly complex system. As we know quite well, complex systems fail, they fail often, and any potential failure scenario can and will happen at some point in the microservice's lifetime. Microservices don't live in isolation, but within dependency chains as part of a larger, incredibly complex microservice ecosystem. The complexity scales linearly with the number of microservice in the overall ecosystem, and ensuring the availability of not only an individual microservice, but the ecosystem as a whole, requires that we impose yet another production-readiness standard onto each microservice. Every microservice within the ecosystem must be *fault tolerant* and *prepared for any catastrophe*.

A fault-tolerant, catastrophe-prepared microservice is one that can withstand both internal and external failures. Internal failures are those that the microservice brings on itself: for example, code bugs that aren't caught by proper testing lead to bad deploys, causing outages that affect the entire ecosystem. External catastrophes, such as datacenter outages and/or poor configuration management across the ecosystem, lead to outages that affect the availability of every microservice and the entire organization.

Failure scenarios and potential catastrophes can be quite adequately (though not exhaustively) prepared for. Identifying failure and catastro-

phic scenarios is the first requirement of building a fault-tolerant, production-ready microservice. Once these scenarios have been identified, the hard work of strategizing and planning for when they will occur begins. This has to happen at every level of the microservice ecosystem, and any shared strategies should be communicated across the organization so that mitigation is standardized and predictable.

Standardization of failure mitigation and resolution at the organizational level means that incidents and outages of individual microservices, infrastructure components, or the ecosystem as a whole need to be wrapped into carefully executed, easily understandable procedures. Incident response procedures need to be handled in a coordinated, planned, and thoroughly communicated manner. If incidents and outages are handled in this way, and the structure of incident response is well defined, organizations can avoid lengthy downtimes and protect the availability of the microservices. If every developer knows exactly what they are supposed to do in an outage, knows how to mitigate and resolve problems quickly and appropriately, and knows how to escalate if an issue is beyond their capabilities or control, then the time to mitigation and time to resolution drop drastically.

Making failures and catastrophes predictable means going one step further after identifying failure and catastrophe scenarios and planning for them. It means forcing the microservices, the infrastructure, and the ecosystem to fail in any and all known ways to test the availability of the entire system. This is accomplished through various types of resiliency testing. Code testing (including unit tests, regression tests, and integration tests) is the first step in testing for resiliency. The second step is load testing, where microservices and infrastructure components are tested for their ability to handle drastic changes in traffic. The last, most intense, and most relevant type of resiliency testing is chaos testing, in which failure scenarios are run (both scheduled and randomly) on production services to ensure that microservices and infrastructure components are truly prepared for all known failure scenarios.

The requirements of building a fault-tolerant microservice that is prepared for any catastrophe are:

- Potential catastrophes and failure scenarios are identified and planned for.
- Single points of failure are identified and resolved.
- Failure detection and remediation strategies are in place.
- It is tested for resiliency through code testing, load testing, and chaos testing.
- Traffic is managed carefully in preparation for failure.
- Incidents and outages are handled appropriately and productively.

The details of production-ready fault tolerance and catastrophe-preparedness requirements are covered in [*Chapter 5, Fault Tolerance and Catastrophe-Preparedness*](#).

Performance

In the context of the microservice ecosystem, scalability (which we covered in brief detail earlier), is related to how many requests a microservice can handle. Our next production-readiness principle—*performance*—refers to how well the microservice handles those requests. A performant microservice is one that handles requests quickly, processes tasks efficiently, and properly utilizes resources (such as hardware and other infrastructure components).

A microservice that makes a large number of expensive network calls, for example, is not performant. Neither is a microservice that processes and handles tasks synchronously in cases when asynchronous (nonblocking) task processing would increase the performance and availability of the service. Identifying and architecting away these performance problems is a strict production-readiness requirement.

Similarly, dedicating a large number of resources (like CPU) to a microservice that doesn't utilize it is inefficient. Inefficiency reduces performance: if it's not clear at the microservice level in every case, it's painful and costly at the ecosystem level. Underutilized hardware resources affects the

bottom line, and hardware is not cheap. There's a fine line between underutilization and proper capacity planning, and so the two must be planned and understood together in order for the availability of the microservice to not be compromised and the cost of underutilization reasonable.

PERFORMANCE REQUIREMENTS

The requirements of building a performant microservice are:

- Appropriate service-level agreements (SLAs) for availability
- Proper task handling and processing
- Efficient utilization of resources

The details of production-ready performance requirements are covered in [*Chapter 4, Scalability and Performance*](#).

Monitoring

Another principle necessary for guaranteeing microservice availability is proper microservice *monitoring*. Good monitoring has three components: proper logging of all important and relevant information; useful graphical displays (dashboards) that are easily understood by any developer in the company and that accurately reflect the health of the services; and alerting on key metrics that is effective and actionable.

Logging belongs and begins in the codebase of each microservice. Determining precisely what information to log will differ for each service, but the goal of logging is quite simple: when faced with a bug—even one from many deployments in the past—you want and need your logging to be such that you can determine from the logs exactly what went wrong and where things fell apart. In microservice ecosystems, the versioning of microservices is discouraged, so you won't have a precise version to refer to in which to find any bugs or problems. Code is revised frequently, deployments happen multiple times per week, features are added constantly, and dependencies are ever-changing, but logs will stay the same, preserving the information needed to pinpoint any problems. Just make sure your logs contain the information necessary to determine possible problems.

All key metrics (such as hardware utilization, database connections, responses and average response times, and the status of API endpoints) should be graphically displayed in real time on an easily accessible dashboard. Dashboards are an important component of building a well-monitored, production-ready microservice: they make it easy to determine the health of a microservice with one glance and enable developers to detect strange patterns and anomalies that may not be extreme enough to trigger alerting thresholds. When used wisely, dashboards allow developers to determine whether or not a microservice is working correctly simply by looking at the dashboard, but developers should never need to watch the dashboard in order to detect incidents and outages, and rollbacks to stable previous builds should be fully automated.

The actual detection of failures is accomplished through alerting. All key metrics must be alerted on, including (at the very least) CPU and RAM utilization, number of file descriptors, number of database connections, the SLA of the service, requests and responses, the status of API endpoints, errors and exceptions, the health of the service's dependencies, information about any database(s), and the number of tasks being processed (if applicable).

Normal, warning, and critical thresholds need to be set for each of these key metrics, and any deviation from the norm (i.e., hitting the warning or critical thresholds) should trigger an alert to the developers who are on call for the service. Thresholds should be signal-providing: high enough to avoid noise, but low enough to catch any and all real problems.

Alerts need to be useful and actionable. A nonactionable alert is not a useful alert, and a waste of engineering hours. Every actionable alert—that is, *every* alert—should be accompanied by a runbook. For example, if an alert is triggered on a high number of exceptions of a certain type, then there needs to be a runbook containing mitigation strategies that any on-call developer can refer to while attempting to resolve the problem.

The requirements of building a properly monitored microservice are:

- Proper logging and tracing throughout the stack
- Well-designed dashboards that are easy to understand and accurately reflect the health of the service
- Effective, actionable alerting accompanied by runbooks
- Implementing and maintaining an on-call rotation

The details of production-ready monitoring requirements are covered in [*Chapter 6, Monitoring*](#).

Documentation

Microservice architecture carries the potential for increased technical debt—it's one of the key trade-offs that come with adopting microservices. As a rule, technical debt tends to increase *with* developer velocity: the more quickly a service can be iterated on, changed, and deployed, the more frequently shortcuts and patches will be put into place. Organizational clarity and structure around the *documentation* and *understanding* of a microservice cut through this technical debt and shave off a lot of the confusion, lack of awareness, and lack of architectural comprehension that tend to accompany it.

Reducing technical debt isn't the only reason to make good documentation a production-readiness principle: doing so would make it somewhat of an afterthought (an important afterthought, but an afterthought nonetheless). No, just like each of the other production-readiness standards, documentation and its counterpart (understanding) directly and measurably influence the availability of a microservice.

To see why this is true, we can think about how teams of developers work together and share their knowledge and understanding of a microservice. You can do this yourself by sitting one of your development teams in a room, in front of a whiteboard, and asking them to sketch the architecture and all important details of the service. I promise you will be surprised by the result of this exercise, and you will most likely find that knowledge and understanding of the service is not cohesive or coherent

across the group. One developer will know one thing about the application that nobody else does, while a second developer will have such a different understanding of the microservice that you will wonder if they are even contributing to the same codebase. When it's time for code changes to be reviewed, technologies to be swapped, or features to be added, the lack of alignment of knowledge and understanding will lead to the design and/or evolution of microservices that are not production-ready, containing serious flaws that undermine the service's ability to reliably serve production traffic.

This confusion and the problems that it creates can be successfully and rather easily avoided by requiring that every microservice follow a very strictly standardized set of documentation requirements. Documentation needs to contain all the essential knowledge (facts) about a microservice, including an architecture diagram, an onboarding and development guide, details about the request flow and any API endpoints, and an on-call runbook for each of the service's alerts.

Understanding of a microservice can be accomplished in several ways. The first is by doing the exercise I just mentioned: stick the development team in a conference room, and ask them to whiteboard the architecture of the service. Thanks to our old friend, the ever-present increased developer velocity, microservices change radically at different times throughout their lifecycle. By making these architecture reviews part of each team's process and scheduling them regularly, you can guarantee that knowledge and understanding about any changes in the microservice will be disseminated to the entire team.

To cover the second aspect of microservice understanding, we need to jump up by one level of abstraction and consider the production-readiness standards themselves. A great deal of microservice understanding is captured by determining whether a microservice is production-ready and where it stands with regard to the production-readiness standards and their individual requirements. This can be accomplished in a myriad of ways, one of which is running audits of whether a microservice meets the requirements, and then creating a roadmap for the service detailing how to bring it to a production-ready state. Checking the requirements can also be automated across the organization. We'll dive into other aspects of this in more detail in the next section on the implementation of produc-

tion-readiness standards in an organization that has adopted microservice architecture.

DOCUMENTATION REQUIREMENTS

The requirements of building a well-documented microservice are:

- Thorough, updated, and centralized documentation containing all of the relevant and essential information about the microservice
- Organizational understanding at the developer, team, and ecosystem levels

The details of production-ready documentation requirements are covered in [*Chapter 7, Documentation and Understanding*](#).

Implementing Production-Readiness

We now have a set of standards that apply to every microservice in any microservice ecosystem, each with its own set of specific requirements. Any microservice that satisfies these requirements can be trusted to serve production traffic and guarantee a high level of availability.

Now that we have the production-readiness standards, the question that remains is *how* we can implement them in a specialized, real-world microservice ecosystem. Going from principle to practice and applying theory to real-world applications always presents us with some significant level of difficulty. However, the power of these production-readiness standards and the requirements they impose lies in their remarkable applicability and strict granularity: they are both general enough to apply to any ecosystem, yet specific enough to provide concrete strategies for implementation.

Standardization requires buy-in from all levels of the organization, and must be adopted and driven both from the top-down and from the bottom-up. At the executive and leadership (managerial and technical) levels, these principles need to be driven and supported as architectural requirements for the engineering organization. On the ground floor, within individual development teams, standardization needs to be embraced

and implemented. Importantly, standardization needs to be seen and communicated not as a hindrance or gate to development and deployment, but as a guide for production-ready development and deployment.

Many developers may resist standardization. After all, they may argue, isn't the point of adopting microservice architecture to provide greater developer velocity, freedom, and productivity? The answer to these sorts of objections is not to deny that the adoption of microservice architecture brings freedom and velocity to development teams, but to agree and point out that that is *exactly* why production-readiness standards need to be in place. Developer velocity and productivity grind to a halt whenever an outage brings a service down, whenever a bad deploy compromises the availability of a microservice's clients and dependencies, whenever a failure that *could have been avoided with proper resiliency testing* brings the entire microservice ecosystem down. If we've learned anything in the past 50 years about software development, we've learned that standardization brings freedom and reduces entropy. As Brooks says in *The Mythical Man-Month*, perhaps the greatest collection of essays on the practice of software engineering, "form is liberating."

Once the engineering organization has adopted and agreed to follow production-readiness standards, the next step is to evaluate and elaborate on each standard's requirements. The requirements presented here and detailed throughout this book are very general and need the addition of context and organization-specific details and implementation strategies. What needs to be done is to work through each production-readiness standard and its requirements and to figure out how each requirement can be implemented in the engineering organization. For example, if the organization's microservice ecosystem has a self-service deployment tool, then implementing a stable and reliable deployment process needs to be communicated in terms of the Support internal deployment tool and how it Sign Out works. Rebuilding internal tools and/or adding features to them may also come out of this exercise.

[TERMS OF SERVICE](#) [PRIVACY POLICY](#)

The actual implementation of the requirements and determining whether or not a given microservice meets them can be done by the developers themselves, by team leads, by management, or by operations (systems, DevOps, or site reliability) engineers. At both Uber and the several other companies I know that have adopted production-readiness standardization, the implementation and enforcement of the production-readiness

standards is driven by the site reliability engineering (SRE) organizations. Typically, SREs are responsible for the availability of the services, and so driving these standards across the microservice ecosystem fits in quite well with existing responsibilities. That isn't to say that the developers or development teams have no responsibility for ensuring their services are production-ready; rather, SREs inform, drive, and enforce production-readiness within the microservice ecosystem, and the responsibility of implementation falls on both the SREs embedded within development teams and on the developers themselves.

Building and maintaining a production-ready microservice ecosystem is not an easy challenge to undertake, but the rewards are great, and the impact can be seen so clearly in the increased availability of each microservice. Implementing production-readiness standards and their requirements provides measurable results, and means that development teams can work knowing that the services they depend on are trustworthy, that they are stable, reliable, fault tolerant, performant, monitored, documented, and prepared for any catastrophe.